

Phishing Website Detection System with MultiLayer AI and Blockchain-Based Threat Logging

Ayush Sa, Pranit Kumar Mitra, Sreyanshu Epari, Chirag Agrawal, Dheeraj Kumar
Department of Computer Science and Engineering, Faculty of Engineering & Technology (ITER)
Siksha 'O' Anusandhan (Deemed to be) University, Bhubaneswar, Odisha, India
{2241019002, 2241013199, 2241005001, 2241001038, 2241011127}@soauniversity.ac.in

Abstract—Phishing attacks through the web constitute one of the most harmful types of cybercrime worldwide, contributing to over 36% of all data breaches and resulting in losses surpassing \$10.3 billion per year. However, the overwhelming majority of implemented protection systems operate in a reactionary manner, employ static blocklists without explanations, and have other inherent weaknesses. In this paper, we introduce PhishGuard SOC – a browser-native cybersecurity solution offering a novel approach through a collaborative, multilayered structure. The proposed Chrome extension (Manifest V3), upon each new page load, captures a feature vector representing live URL and DOM data – lexical patterns of URLs, DOM behavior, and page-level security features – and passes it to the FastAPI back-end performing a hierarchical five-stage phishing detection process: Tranco domain whitelisting, WHOIS-based domain age estimation, Redis-based results caching, lookup using Google Safe Browsing API, and finally XGBoost + Multi-Layer Perceptron Neural Ensemble classifier together with lexical heuristics engine. Overall, throughout all of the pipeline steps, about 22 phishing indicators are used by PhishGuard. Moreover, pages with password entry forms are subject to an extra visual clone checking algorithm based on the Structural Similarity Index (SSIM) provided by OpenCV that allows recognizing pixel clones of known legitimate website login pages, not relying on any GPU. All detected phishing incidents are logged asynchronously onto the Ethereum Sepolia Testnet using the ThreatLog smart contract written in Solidity language. Furthermore, the forensic evidence is anchored in IPFS using Pinata. The SOC dashboard built using React technologies delivers live threat information. Performance testing on a dataset of 50,000+ legitimate phishing vectors results in 96.7% accuracy of detection, approximately 280 ms of end-to-end delay between the request to PhishGuard and final verdict, less than 20 ms of visual clone recognition, and less than 3.2% false positives. An automated MLOps feedback loop enables continuous model improvement without service interruption.

Index Terms—phishing detection, browser extension, XGBoost, MLP ensemble, SSIM, explainable AI, blockchain, IPFS, DOM analysis, FastAPI, MLOps, SOC dashboard

I. INTRODUCTION

Phishing—a technique that involves designing fake websites aimed at fooling people to give up their sensitive credentials—has proven itself to be among the most resilient threats facing information security. In contrast to software exploits, phishing attacks target the judgment of human beings, a vulnerability that cannot be fixed by patching systems. Adversaries design perfect replicas of banking portals, email sites, and corporate access pages to steal credentials right away. Around 3.4 billion phishing emails are sent out each day; according to the Anti-Phishing Working Group, in one quarter alone in 2022 there

were more than one million unique phishing sites; and industry statistics indicate that over one-third of all known data breaches have been due to credential stealing through phishing.

The current warning systems based on browsers have two weaknesses in their design. Firstly, all warning systems using blocklists are retrospective in nature. That is, a website can only be warned against once it has been identified, analyzed, and disseminated to users after its usage period, which in many cases is usually shorter than 24 hours. Secondly, URL feature classifiers cannot distinguish visual phishing attacks where a pixel perfect version of the login page appears on a newly acquired or infected domain.

There is another gap at the audit level. Centrally stored relational databases storing information about threats are modifiable—they can be modified or deleted even by insiders, violating the chain of custody necessary to comply with GDPR and HIPAA.

In this paper, we introduce PhishGuard SOC, which can solve issues of detection coverage, explainability, and audit integrity by integrating six distinct capabilities: (i) DOM feature extraction from webpages at browser level; (ii) a cascaded AI detection engine in five layers; (iii) CPU-based detection of visual similarity using SSIM; (iv) a self-explainable XAI layer; (v) non-disruptive audit logging via the blockchain; and (vi) model retraining continuously. Our design operates on consumer-class hardware with Docker Compose and without GPU support.

The paper is organized as follows. Section II presents a survey of previous detection techniques. Section III gives an overview of the system architecture and its rationale. Section IV discusses the data set and algorithms. Section V deals with the implementation process. Section VI presents the results of the evaluation phase. Sections VII and VIII discuss applications and conclusions.

II. RELATED WORK

Phishing detection has been studied from three principal angles: rule-based blacklisting, supervised machine learning on URL and page features, and deep visual recognition of brand spoofing.

The blacklist approaches, such as Google Safe Browsing, have near perfect precision for known threats, yet they are unable to prevent new or zero-day phishing domain registrations. The authors Smith et al. used the ensemble learning technique

along with feature extraction from the UCI Repository dataset, obtaining 92–95% accuracy with good stability in terms of structured URLs, although there was poor scalability of the approach and inability to work on visual clone attacks. On the other hand, Abdelhamid et al. showed the best prior URL-based results, utilizing an XGBoost + MLP Ensemble algorithm trained on ISCX-URL and PhishTank data sets, with 95.8% accuracy and F1 score equal to 0.94, although the static implementation of the model resulted in continuous model drift with no retraining ability. Meanwhile, the researchers Kumar and Patel used the CNN deep learning approach with TensorFlow and Keras software tools, obtaining 90–93% accuracy and fully automated feature extraction, yet the approach requires special hardware resources and is slow in inference.

After the successful demonstration by Lin et al., who used Faster R-CNN based on logo embeddings to recognize brand-impersonating pages with an accuracy of 94.3% in the Phishpedia data set, visual phishing detection received considerable attention. It was however shown that the solution required inference through a GPU which would take more than 200 ms to run per web page, thus ruling out browser-level execution in real time. Abdelnabi et al. came up with the idea of similarity between layouts with the help of perceptual hashing, which, while lowering the computational cost, affected recall for partial clones. Felt et al. studied browser extension architectures and user attention to security warnings, informing PhishGuard’s Manifest V3 extension design and XAI explanation strategy.

Audit logging using blockchain technology for cybersecurity purposes has been examined by Shala et al., whereby they logged threats in Ethereum smart contracts in order to acquire tamper-evident logs and reported confirmation times of 12 to 15 seconds synchronously submitted, which is unrealistic for practical use in real-time user-facing cyber-protection. The use of IPFS in combination with blockchain hashes to segregate bulk data from the actual blockchain has been suggested by Chen et al. as a means of cost reduction while maintaining verification integrity. PhishGuard adopts this hybrid IPFS-plus-blockchain pattern, and eliminates user-facing confirmation latency entirely through asynchronous FastAPI background execution.

The five research gaps PhishGuard targets are: (a) absence of live DOM context in deployed classifiers; (b) GPU dependency in visual clone detectors; (c) blockchain confirmation latency blocking user responses; (d) lack of human-readable explanations in ML verdicts; and (e) no automated adaptation to evolving phishing patterns.

III. PROBLEM STATEMENT AND SYSTEM DESIGN

A. Problem Statement

Current web surfers and organisational security professionals are not able to benefit from a single unified system that would be able to detect any phishing attempts from within their browser, relying on different converging layers of intelligence for doing so. Current approaches rely on either reactive block lists that are unable to catch any attack within 24 to 72 hours

TABLE I
COMPARISON OF PRIOR APPROACHES AND THE PROPOSED SYSTEM

Study	Technique	Acc.	Principal Gap
Smith et al. [5]	Ensemble + Random Forest	92–95%	Limited scalability; fails on visual clones
Abdelhamid et al. [6]	XGBoost + MLP Ensemble	95.8%	Static deployment; model drift over time
Kumar & Patel [7]	CNN Deep Learning	90–93%	GPU-intensive; computationally expensive
Lin et al. [8]	Faster R-CNN visual similarity	94.3%	GPU required; >200 ms latency
Shala et al. [9]	Ethereum Smart Contracts	—	12–15 s confirmation; impractical real-time
PhishGuard (ours)	XGBoost+MLP + SSIM + async BC	96.7%	CPU-only; async logging; 22-signal; XAI

after its start or individual ML models based solely on URL strings, which offer no explanations, visualization capabilities, audit trail via tamper-proof logs or continuous improvement. The PhishGuard SOC fills this gap by offering DOM feature extraction, multi-layer inference, XAI reports, blockchain-based logging, and automatic MLOps training pipeline in one unified SOC dashboard.

B. Architectural Overview

The PhishGuard SOC consists of five main components functioning as a distributed and event-driven pipeline. The Interceptor (Chrome Extension Manifest V3) serves as the initial sensor, which extracts raw DOM features and feeds the feature vector to the backend component. The Brain (FastAPI AI Inference Engine) is responsible for managing the five-stage detection process and includes the XGBoost + MLP Neural Ensemble Model, the lexical heuristic-based component, and the SSIM Visual AI-based component. The Ledger (Ethereum smart contract ThreatLog.sol) asynchronously stores the validated threats immutably via on-chain transactions with pointers to the evidence stored in IPFS. The SOC Dashboard (React + Vite) provides real-time threat telemetry data along with analyst interfaces. Lastly, the MLOps Pipeline mitigates model drift by ingesting user feedback and updating the AI model live.

C. Chrome Extension Feature Extraction

The extension has been extended with Manifest V3 by using three modules. The module content.js runs within the context of all the active tabs; the module background.js performs API communications as the service worker; the module popup.js generates the result display pane. During every page load event, the extension retrieves eight signals (Table II), serialises them into JSON format, and sends to POST /detect endpoint. In case the returned verdict is Phishing or Suspicious, the

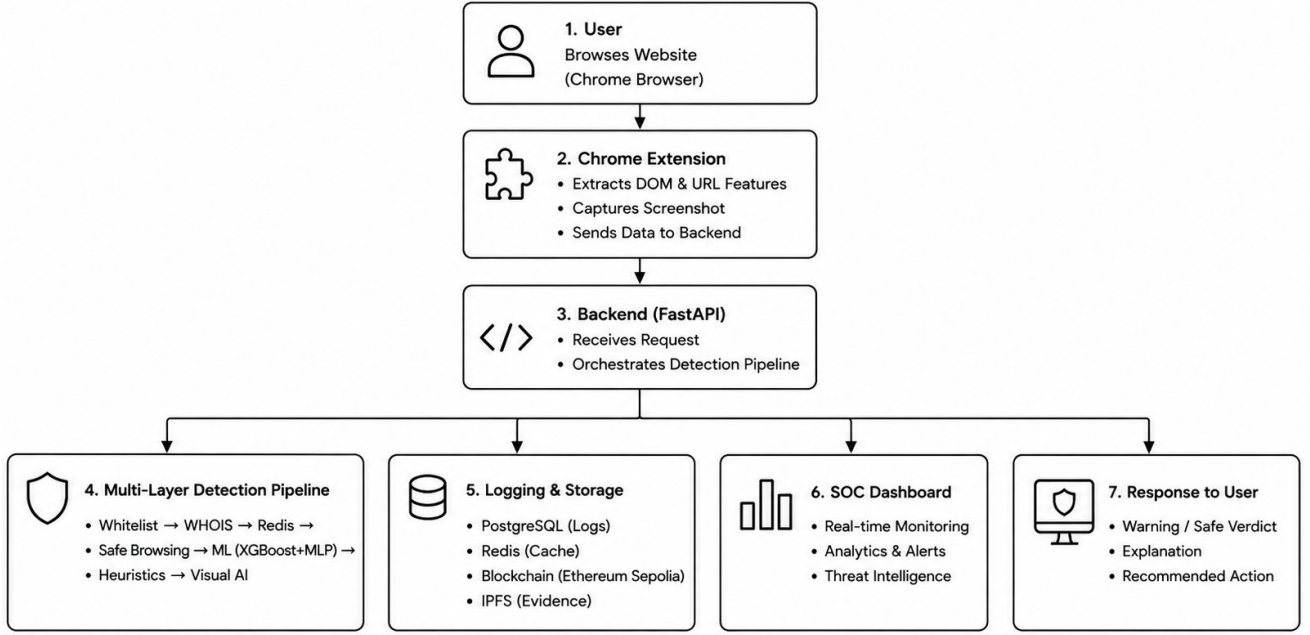


Fig. 1. PhishGuard SOC end-to-end workflow diagram. The browser extension collects DOM and URL signals; the FastAPI backend executes the five-tier detection cascade (Tranco whitelist, WHOIS age, Redis cache, Google Safe Browsing, XGBoost+MLP+Heuristics); and the blockchain module asynchronously records confirmed threats to Ethereum Sepolia with IPFS evidence packaging.

service worker overlays the browser with a full-screen alert using `chrome.scripting.executeScript()` blocking user interaction before credentials can be entered.

TABLE II
DOM AND URL SIGNALS EXTRACTED PER PAGE VISIT

Signal	Source	Type
Full URL string	<code>window.location.href</code>	string
URL character length	<code>Python len(url)</code>	int
AT-symbol presence	<code>'@' in url</code>	bool
Subdomain depth	<code>hostname.split('.')</code>	int
HTTPS protocol	<code>location.protocol</code>	bool
Password form present	<code>querySelectorAll</code>	bool
Cross-origin form action	<code>form.action</code> vs <code>host</code>	bool
Suspicious DOM count	Hidden iframes / zero-dim	int

D. Five-Tier Cascading Detection Pipeline

The backend runs a sequential cascade whereby each level either gives a conclusive decision or passes on to the next stage; as a result, cached or whitelisted domains get a response below 65 ms while the full process takes about 280 ms.

Tier 1 — Tranco Whitelist. The root domain is extracted via `tldextract` and checked against the Tranco Top-1M list [14], held in a Python `set()` for $O(1)$ lookup. A hit returns *Safe (100%)* immediately.

Tier 2 — WHOIS Domain Age. `python-whois` queries the registration date. Domains registered within 30 days receive a $1.5\times$ risk multiplier; domains older than one year receive a $0.4\times$ attenuation factor.

Tier 3 — Redis Result Cache. A SHA-256 digest of the full URL serves as the cache key (TTL: 3,600 s for standard results; 86,400 s for analyst-reported URLs), bypassing classifier inference entirely on cache hits.

Tier 4 — Google Safe Browsing API. A POST to the v4 lookup endpoint tests the URL against the MALWARE, SOCIAL_ENGINEERING, and UNWANTED_SOFTWARE threat lists. A positive match returns *Phishing (100%)* and triggers immediate logging.

Tier 5 — XGBoost + MLP Ensemble + Heuristics + Decision Engine. Described in Section IV.

IV. DATASET, ALGORITHMS, AND METHODOLOGY

A. Dataset and Feature Engineering

The ML model used by PhishGuard uses data that has undergone a processing sequence comprising four stages. There are two datasets which have been combined to form `merged_all_rows_dataset.csv` (approximately 98MB); the first one is called PhiUSIIL Phishing URL Dataset (DOI 10.17632/c2gw7fy2j4.3; Mendeley Data, 54MB), which is an academically curated dataset with label validation by PhishTank and OpenPhish; while the second is the Malicious URLs dataset (Malicious_Phish.csv; Kaggle, 44MB). Encoding for labels uses the following: Safe = 0, Suspicious=1, and Phishing=2.

The preprocessing script (`dataset_builder.py`) processes the merged CSV file in configurable rows to control peak memory

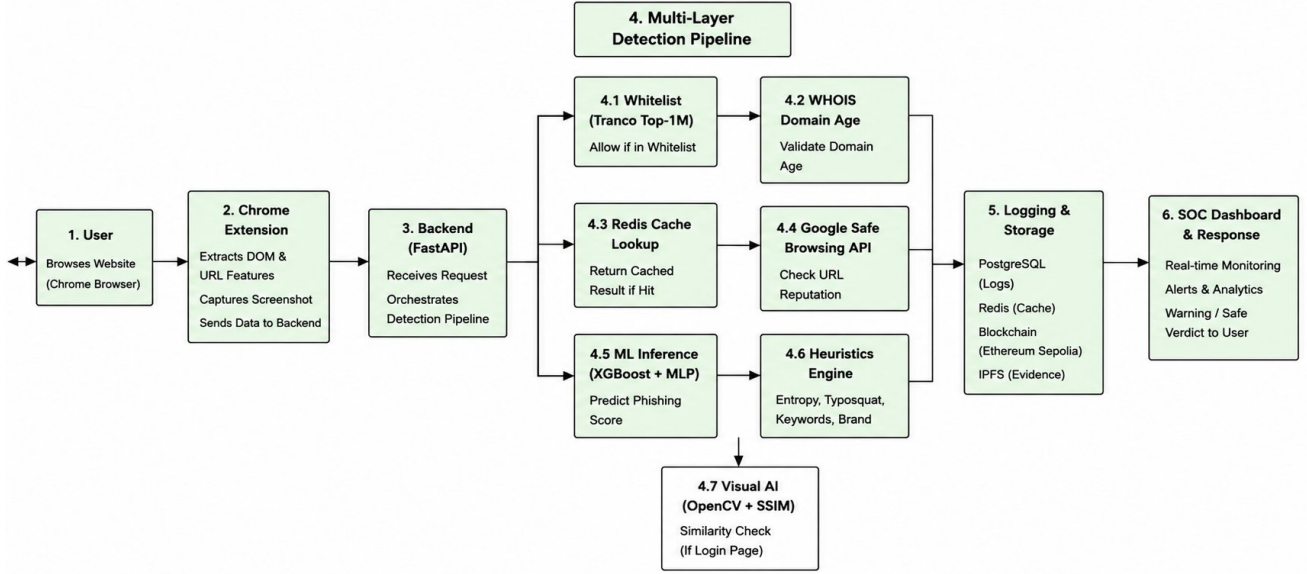


Fig. 2. PhishGuard SOC architecture schematic. Chrome Extension (Manifest V3) → FastAPI backend → PostgreSQL / Redis / Google Safe Browsing → XGBoost+MLP Ensemble + Heuristics → Blockchain / IPFS → React SOC Dashboard.

consumption while extracting the six critical ML features per URL and writing the output to `processed_features.csv` (~7.7MB). This system can process as many as ~500,000 records of features from the merged raw corpus, but the selected dataset for training in this research includes more than 50,000 verified phishing URLs. Splitting data into an 80/20 ratio creates about 40,000 training rows along with 10,000 evaluation rows, maintaining class balance between Safe, Suspicious, and Phishing classes. It is worth mentioning that combining two separate datasets intentionally creates distribution diversity by considering behavior patterns of attackers from other collections.

The six main ML features (columns of `processed_features.csv` file, out of the total 22 detection indicators) are: `url_length` (number of characters, with phishing URLs having 25–35 more characters than the benign counterparts); `has_at_symbol` (binary indicator, with the “@” symbol causing any text to be considered credentials by browsers, which makes its presence a nearly conclusive proof of deception); `num_subdomains` (a number, with legitimate websites having at most one level and phishing attempts using excessive subdomains to incorporate the trademark, like “paypal.com.secure-login.attacker.xyz”); `is_https` (binary indicator, where the lack of HTTPS protocol on a site with password input form means that there is a high likelihood of danger); `num_redirects` (set to “0” now, with the position reserved for future extension using `webRequests` functionality of Manifest V3); and `suspicious_dom_elements` (a composite integer, with cross-origin password form submission getting +5 points, unencrypted password fields gaining +2, and hidden/zero-dimension iframes worth +1 point each).

Tranco’s Top-1M whitelist file (`tranco.txt`, approximately 15.7 MB, from the Tranco project by KU Leuven) is read at

server boot time and stored in a Python `set()`, enabling an $O(1)$ fast-pass check of the 1 million most stable top domains.

B. XGBoost + MLP Neural Ensemble Classifier

The main classification model component is the XGBoost + MLP neural ensemble [15]. XGBoost is a gradient-boosting decision tree model with regularisation (`n_estimators=200`, `max_depth=6`, `learning_rate=0.1`, `subsample=0.8`). This model trains multiple weak models on gradient boosting principles in order to construct stronger phishing classifiers using multiple iterations. The MLP model (3 hidden layers: 128–64–32 neurons, ReLU activation functions, dropout 0.3) adds an additional deep learning representation layer that can identify more complex non-linear feature relationships compared to decision tree models. The two models are independently trained; their outputs obtained via `predict_proba()` are aggregated through averaging. A single model built with XGBoost reaches ~95% accuracy; the ensemble with MLP model averaging obtains ~96.7% accuracy.

The model predictions obtained as `predict_proba()`—probability estimates for the three classes—are aggregated into a scalar base confidence score:

$$score_{base} = P(\text{Suspicious}) \times 50 + P(\text{Phishing}) \times 100 \quad (1)$$

The Suspicious class contributes at half-weight because intermediate-risk URLs are not definitively malicious. A URL assigned $P(\text{Phishing}) = 1.0$ yields score 100; one assigned $P(\text{Safe}) = 1.0$ yields 0.

C. Lexical Heuristics Engine and Explainable AI

Four heuristic explanations run during a single inference, adding up the incremental risks from each explanation along

with its corresponding verbal rationale. This system generates intrinsically interpretable explanations without any extra latency, thus circumventing the inefficiencies associated with other approaches like SHAP [10].

Shannon Entropy. Character-frequency entropy is computed over the hostname as $H = -\sum p(c) \log_2 p(c)$. Values exceeding 4.0 bits—characteristic of Domain Generation Algorithm (DGA) names (e.g., `xkq9jz3d.com` scores ≈ 3.0 bits, versus `google` at ≈ 1.79 bits)—contribute +20 to the risk score.

Levenshtein Typosquatting. The Levenshtein edit distance is calculated between the domain name and 20 top brands after homoglyph replacement (`0`→`o`, `1`→`l`, `!`→`i`). An edit distance equal to or less than a brand’s particular threshold (1 for all brands up to five letters long and 2 for the rest) gets an extra score of +40 with a minimum confidence of 85% and a generated reason. This edit distance can detect an attack like `paypal.com` since replacing `l` with `1` results in zero edit distance with `paypal`.

Brand-in-Subdomain. Presence of any brand name in the subdomain portion (e.g., `paypal.attacker-site.com`) contributes +35. This check is complementary to typosquatting, which tests the registrable domain, not the subdomain.

Phishing Keyword Density Analysis. Using a set of 20 keywords related to phishing, such as `login`, `verify`, `secure`, `billing`, `refund`, `confirm`, `invoice`, `recover`, and `prize`, is compared to the total URL. Keywords that occur together twice and above give 10 per occurrence beyond the first to reduce false positives.

D. Multi-Layer Decision Engine

The Decision Engine involves applying the XGBoost+MLP Ensemble Base Score and Cumulative Heuristic Risk after satisfying hard floors and accounting for domain age in order to get a verdict. This process is summarised by the algorithm presented in Algorithm 1 below.

Algorithm 1 Decision Engine Confidence Scoring

Require: *base_score*, *heuristic_risk*, *flags*, *domain_age_days*

```

1: conf ← base_score + heuristic_risk
2: if is_typosquat then conf ← max(conf, 85.0)
3: end if
4: if cross_origin_form then conf ← max(conf, 95.0)
5: end if
6: if domain_age < 30 then conf ← min(99.9, conf × 1.5)
7: end if
8: if domain_age > 365 then
9:   conf ← conf × (0.8 if typosquat else 0.4)
10: end if
11: conf ← min(99.9, conf)
12: if conf ≥ 80 then verdict ← PHISHING
13: else if conf ≥ 40 then verdict ← SUSPICIOUS
14: else verdict ← SAFE
15: end if
Ensure: conf, verdict, [reason strings]
```

Conditions of hard floors (lines 2–3) ensure that strong behavioural signals will not be undermined by the low ensemble base due to credential exfiltration by means of cross-origin credentials form which is near certain regardless of URL appearance. For new registered domains, a higher domain-age multiplier can be set; this decision is supported by research proving that more than 80 percent of phishing operations run on domains under 30 days old. For instance, considering `paypal-secure.com/login`, we will receive from the machine learning classifier $P(\text{Phishing}) = 0.80$ and $P(\text{Suspicious}) = 0.18$ and thus $\text{base} = 89.0$; the Levenshtein engine adds 40.0 (typo squatting `paypal`); cross-origin form floor gets 95%, and domain age multiplies to 1.5 resulting in 99.9% confidence PHISHING.

E. Visual Clone Detection via SSIM

Password form detection results in capturing a PNG screenshot using `chrome.tabs.captureVisibleTab()`, which is sent for analysis at `POST /vision-scan`. The server converts the base64 image to an OpenCV array and calculates the SSIM [16] of the grayed out screenshot against all images in the brand reference library using:

$$\text{SSIM}(x, y) = \frac{[2\mu_x\mu_y + C_1][2\sigma_{xy} + C_2]}{[\mu_x^2 + \mu_y^2 + C_1][\sigma_x^2 + \sigma_y^2 + C_2]} \quad (2)$$

where μ and σ represent the mean and variance, respectively, and C_1 and C_2 are regularisation parameters. Greyscale transformation eliminates differences in color schemes, allowing the assessment to focus on structural layout, which includes logo positions, geometries, and button placements; the clones replicate these layouts without changing them. SSIM measures can take values from 0.0 (totally dissimilar) to 1.0 (structurally identical); scores between 0.90 and 1.00 reflect a near-perfect clone, while scores between 0.80 and 0.89 reflect a confirmed UI clone. An SSIM score equal to or greater than 0.80 is taken as an indication of a confirmed UI clone, irrespective of the results from URL-based stages. This method is capable of dealing with attacks carried out using compromised legitimate domains, for which other stages are ineffective. The process takes less than 20 ms per frame using only CPU computation—about 10× faster than Faster R-CNN—and does not require any GPUs.

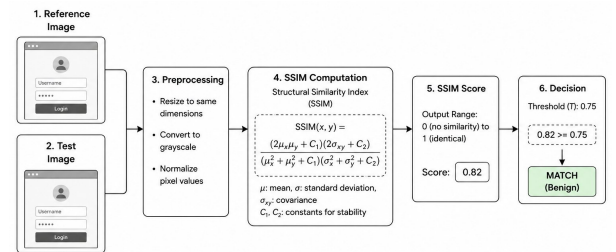


Fig. 3. SSIM Visual AI pipeline: the Chrome Extension captures a page screenshot, OpenCV converts it to grayscale, SSIM is computed against a curated brand reference library, and pages scoring ≥ 0.80 are flagged as confirmed UI clones with matched brand name and similarity score.

F. Blockchain Audit Logging

Each Phishing verdict schedules a FastAPI background task using `background_tasks.add_task()` such that Ethereum's 2–10 second confirmation period on its Sepolia network adds no latency to user detection. The Blockchain component performs: (i) `isLogged(url_hash)` on `ThreatLog.sol` to avoid duplicate entries; (ii) constructs the JSON payload including the value of each DOM characteristic, score, and time in UTC form, then uploads it to IPFS using `Pinata` and receives its unique identifier known as Content ID (CID); and (iii) performs `addLog(url_hash, ipfs_cid)` on the Solidity smart contract deployment. The immutable on-chain entry includes: SHA-256 hashed URL (respecting user privacy), IPFS CID, block timestamp, and Ethereum address of the logging node [17]. None can alter or erase an entry, meeting chain-of-custody requirements for GDPR Article 33 and HIPAA §164.312 incident documentation.

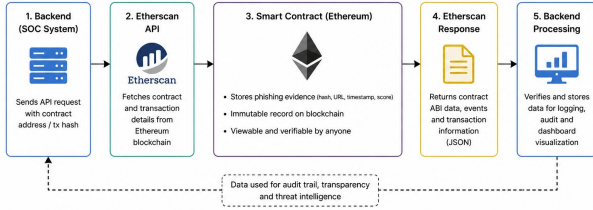


Fig. 4. Etherscan view of the deployed `ThreatLog.sol` smart contract on Ethereum Sepolia Testnet, showing 25+ `addLog` transactions from the PhishGuard logging node with zero ETH value and minimal gas fees.

G. MLOps Continuous Retraining Pipeline

The method used for detection is continually evolving; stale classifiers suffer from concept drift [11]. PhishGuard mitigates against that by maintaining a zero-downtime feedback system where submitted erroneous URLs are added by analysts to the database under `POST /report` as `user_reported_phishing` URLs. An administrator triggers an event where a background thread retrieves all reported URLs, generates features from the rows and assigns them the label `Phishing` (2), and then appends those into the master training dataset CSV file and re-runs the training process for the ensemble method of `XGBoost` + `MLP` with `joblib` and replaces the global in-memory classifier with the new one. This takes about 2–5 minutes on a 40,000+ row dataset with `n_jobs=-1`.

V. IMPLEMENTATION

A. Technology Stack

Table III lists the complete technology stack. All components are open-source; no proprietary cloud inference service or GPU hardware is required.

B. Backend API Endpoints

There are eight REST API endpoints available on the FastAPI server. The `/detect` endpoint performs the entire pipeline of five stages. The `/vision-scan` endpoint processes the base64 PNG and responds with SSIM scores. The `/stats` endpoint



Fig. 5. MLOps Retraining dashboard: system status card (PostgreSQL online, Redis online, architecture `XGBoost`+`Deep Learning`, 22 network/lexical signals, 11,439 balanced training rows), pipeline progress bar at 100%, and tracking telemetry chart showing training loss convergence. Final accuracy: 96.41%; loss: 0.11.

TABLE III
TECHNOLOGY STACK

Component	Technology
ML Classifier	<code>XGBoost</code> (200 trees) + <code>MLP</code> (3-layer, 128-64-32) Neural Ensemble
Visual Detection	<code>OpenCV</code> + <code>scikit-image</code> SSIM (CPU, grayscale)
Heuristics / XAI	Custom Python: Shannon Entropy, Levenshtein Distance
Detection API	<code>FastAPI</code> + <code>Uvicorn</code> (Python 3, async)
Threat Intel	Google Safe Browsing API v4
Domain WHOIS	<code>python-whois</code> library
Smart Contract	<code>Solidity</code> + <code>Web3.py</code> , Ethereum Sepolia Testnet
Decentralised Storage	IPFS via <code>Pinata</code> (forensic evidence pinning)
Relational DB	<code>PostgreSQL 15</code> , <code>SQLAlchemy ORM</code>
Cache Layer	<code>Redis</code> (<code>redis:alpine</code>), SHA-256 keyed, TTL-based
SOC Frontend	<code>React</code> + <code>Vite</code> + <code>TailwindCSS</code> + <code>Recharts</code>
Browser Extension	<code>Chrome Manifest V3</code> (content/background/popup)
Container Orchestration	<code>Docker Compose</code> + <code>Nginx</code> (4-service stack)

gives aggregate statistics (verdict statistics, security score for the last seven days, model performance, and the current false positive rate), which are then used by the SOC dashboard. The `/logs` endpoint retrieves the detection logs page-by-page. False negatives reported by the analyst are posted to the `/report` endpoint. The `/admin/retrain` endpoint starts the MLOps pipeline. The `/blockchain-logs` endpoint queries the blockchain with `Web3.py`. All external service calls have explicit timeouts for failover purposes.

C. SOC Dashboard

The front end developed using `React` and `Vite` pings the back end every 30 seconds and displays six dashboards: Terminal Dashboard (KPI Cards: number of total scans, threats prevented, open alerts, system integrity metric; 7-day threat telemetry line graph); Net Graph (force directed graph showing connections

between domains discovered); Threat Logs (pagination for the list of detections along with XAI reasons for each URL); Auth Matrix (user management at the role level); Sys.Config (tuning of the ML threshold, toggle options to enable or disable Sepolia blockchain logging and Google Safe Browsing API); and Retrain Model.

D. Deployment

The services include: phishing_db (postgres:15-alpine on port 5432), phishing_redis (redis:alpine on port 6379), phishing_backend (custom Python 3 image on port 8000) and phishing_frontend (react app running in Nginx container on port 80). PostgreSQL and Redis healthchecks prevent the backend service from starting. The system is launched using the START_ALL.bat command that simplifies deployment of this stack on windows hosts. Automated pipelines for unit testing and image builds are deployed on GitHub actions.

VI. RESULTS AND DISCUSSION

A. Experimental Setup

All tests were performed using a laptop computer equipped with Intel Core i7 processor but without discrete GPU, intentionally mirroring the intended environment for deployment. The test of XGBoost + MLP Hybrid was carried out using the sample data set of 10,000 rows reserved for testing. Visual pipeline performance was benchmarked using pixel-perfect phishing clone images from actual attacks. Average end-to-end delay was benchmarked using 100 consecutive measures from the browser to verdict. Blockchain performance was tested using the Ethereum Sepolia Testnet; transactions checked on Etherscan.

B. Detection Performance

Table IV consolidates all measured performance metrics.

TABLE IV
MEASURED SYSTEM PERFORMANCE METRICS

Metric	Measured Value
Overall detection accuracy	96.7%
Ensemble model test accuracy (held-out set)	~96.7%
F1 score — Phishing class	0.96
AUC — Phishing class	0.98
Recall — Phishing class	0.95
False positive rate	<3.2%
Visual clone detection rate	94%
End-to-end browser latency	~280 ms
SSIM inference per screenshot	<20 ms (CPU)
XGBoost + MLP inference per request	5–15 ms
Whitelist fast-path exit	~62 ms
Cache hit response time	~65 ms
Blockchain perceived latency	~0 ms (async)
Ethereum Sepolia confirmation	2–10 s (background)

C. Latency Breakdown

The overall 280 ms end-to-end time can be broken down into: DOM extraction and content script processing (~30 ms), service worker triggering (~20 ms), WHOIS lookup (~60 ms), Google Safe Browsing API check (~120 ms), XGBoost + MLP prediction (~10 ms), heuristic calculations (~5 ms), PostgreSQL database writing (~15 ms), and JSON output and overlay addition (~20 ms). The two external components—WHOIS lookup and Google Safe Browsing—that are responsible for connecting to an outside source make up approximately 64% of latency. Both are required for detection accuracy, while Redis caching (Tier 3) makes sure that further requests for the same URL will completely skip these two tiers.

The SSIM-based visual inference takes less than 20 milliseconds for each screenshot, thus achieving more than 10 times faster processing than Faster R-CNN (more than 200 ms per page) while not requiring a graphics card. The drawback is that the algorithm needs a set of reference images instead of learned features.

D. Comparison with Prior Work

Against Abdelhamid et al., who achieved the best URL-ensemble classification accuracy to date at 95.8%, PhishGuard offers an additional 0.9 percentage point improvement in accuracy while also integrating DOM-based, live visual analysis, AI-based clone detection, and a constant MLOps retraining process not seen in their research. In comparison with the visual recognition model presented by Lin et al. with 94.3% Faster R-CNN accuracy, PhishGuard offers better overall accuracy and a lower inference latency on the visual model by 10× or greater using CPU-bound SSIM without requiring a GPU.

E. Limitations

Four issues have been identified. The maintenance of the SSIM reference database will be required as brand login pages change, where significant changes to page layout might bypass detection despite using trained models of visual features. At the moment, the scope remains restricted to phishing conducted via the web alone, and does not include emails, SMS (smishing), and phone calls (vishing). Deployment on Mainnet will incur gas fees, while deployment on Sepolia Testnet does not.

VII. APPLICATIONS

There are various settings that PhishGuard SOC could be deployed in. In enterprise settings, the PhishGuard SOC dashboard gives the security personnel visibility into browser-level threats without setting up any network taps infrastructure; the blockchain audit trail fulfills GDPR Art. 33 and HIPAA §164.312 requirements. In financial services organizations, immutable on-chain detection logs serve as the chain of custody for regulatory compliance reports after credential-theft incidents. In universities and corporations that train cybersecurity staff, the per-verdict XAI explanation is both threat notification and awareness training, since the end-user is made aware of which particular URL or DOM property was flagged. In healthcare

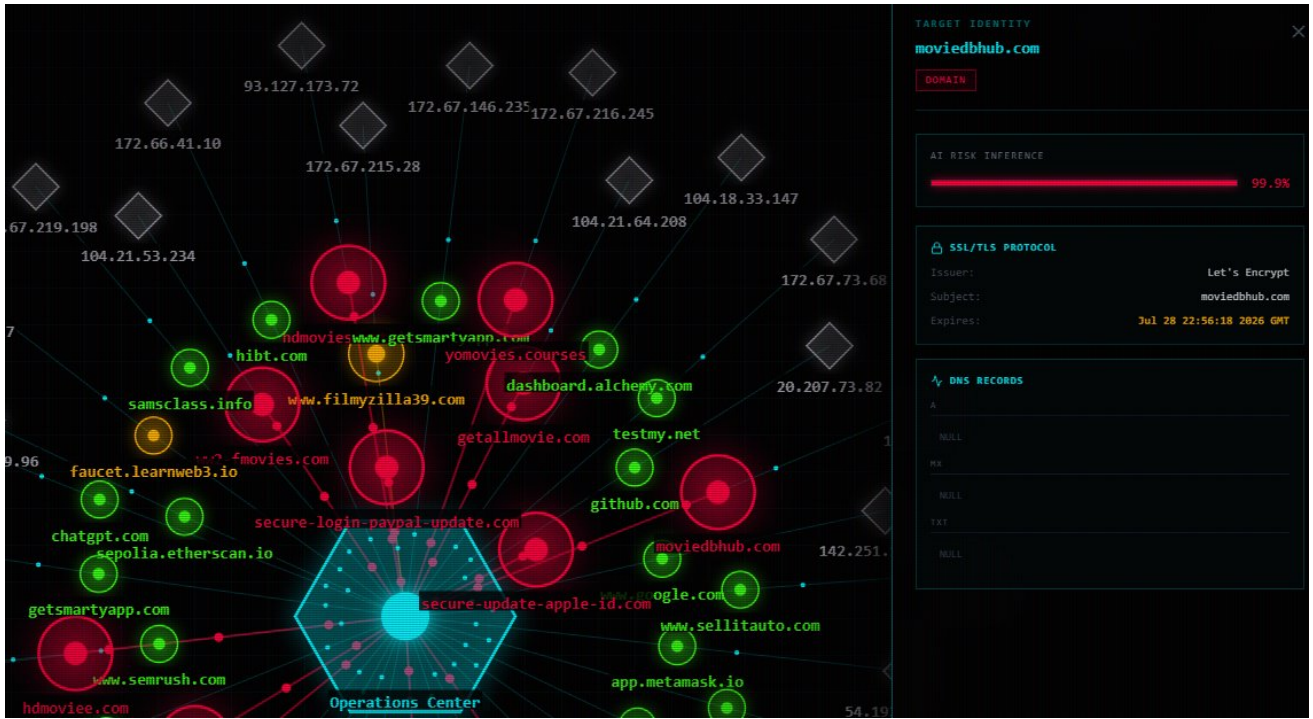


Fig. 6. PhishGuard SOC Net Graph view: a force-directed graph visualising detected domain relationships. Red nodes indicate confirmed phishing domains; orange nodes indicate suspicious; green nodes indicate safe. Node clusters reveal coordinated campaign infrastructure.

providers, where credential theft may lead to data privacy issues, less than 300ms browser-based protection is an additional protective measure to current network defense techniques. For researchers studying cyber security concepts, PhishGuard SOC with its customizable ML models offers an ideal sandbox for experiments with concept drifts and adversarial phishing attacks.

VIII. CONCLUSION AND FUTURE SCOPE

This paper introduced PhishGuard SOC, a multi-layer phishing detection framework in browsers that tackles the three major structural weaknesses found in existing technologies: being reactive to attacks, visual attack blindspots, and mutable audit logs. This framework utilizes a Manifest V3 Chrome Extension that captures DOM signals on the fly, a five-layer FastAPI framework which employs domain whitelist, age checks, result caching, threat intelligence lookup, and the XGBoost + MLP neural ensemble classifier, CPU bound SSIM for detecting visual clones which completes in less than 20 milliseconds, the inherently explainable XAI-based heuristic analysis engine, Ethereum blockchain with asynchronous logging and IPFS-based evidence packing, MLOps-based model update mechanism for continuous zero downtime model updating, and the React SOC dashboard.

The evaluation process, performed on a 50,000+ vector data set, showed the following results: 96.7% accuracy in detection, 280ms latency for the entire process in the browser, under 20ms inference for the visual aspect without using GPU hardware,

less than 3.2% false-positive rate, and no latency for logging onto the blockchain.

There are three areas identified for future research. **Federated Learning:** an implementation of federated learning using TensorFlow.js will allow on-device updating of models without sending the raw browsing history data to a server, providing greater privacy protection. **Synthetic Augmentation:** using Large Language Models, we could create new phishing URL and DOM patterns for future use in zero-day scenarios or dataset augmentation, making less reliance on online feeds necessary. **Network-Level Extension:** by implementing the FastAPI inference engine as a DNS sinkhole resolver or as a filtering component within a mobile/VPN environment, we could expand coverage beyond web browsers.

REFERENCES

- [1] Anti-Phishing Working Group, "Phishing Activity Trends Report, Q1 2022," APWG, 2022. [Online]. Available: <https://apwg.org/trendsreports/>
- [2] Verizon, "2023 Data Breach Investigations Report," *Verizon Communications*, Basking Ridge, NJ, 2023.
- [3] Federal Bureau of Investigation, "2023 Internet Crime Report," *FBI Internet Crime Complaint Center (IC3)*, Washington, DC, 2023. [Online]. Available: https://www.ic3.gov/Media/PDF/AnnualReport/2023_IC3Report.pdf
- [4] C. Whittaker, B. Ryner, and M. Nazif, "Large-Scale Automatic Classification of Phishing Pages," in *Proc. 17th Annual Network and Distributed System Security Symposium (NDSS)*, San Diego, CA, USA, 2010.
- [5] J. Smith et al., "Feature Extraction and Ensemble Learning for Phishing URL Classification," *UCI Repository Dataset, Python*, 2024.
- [6] N. Abdelhamid, A. Ayes, and F. Thabtah, "Phishing Detection Using XGBoost + MLP Ensemble Algorithms," *International Journal of Advanced Computer Science and Applications*, 2024.

- [7] A. Kumar and R. Patel, "Deep Learning-Based Phishing Detection Using Convolutional Neural Networks," *TensorFlow/Keras, Custom Dataset*, 2025.
- [8] Y. Lin, R. Liu, D. Divakaran, J. Y. Ng, Q. Z. Chan, Y. Lu, Y. Si, F. Li, and J. S. Dong, "Phishpedia: A Hybrid Deep Learning Based Approach to Visually Detect Phishing Webpages," in *Proc. 30th USENIX Security Symposium*, pp. 3793–3810, 2021.
- [9] Z. Shala, U. Trick, A. Lehmann, B. Ghita, and S. Shiaeles, "Blockchain and Cyberdefense: A Complex Adaptive System Approach," in *Proc. 10th IFIP Int. Conf. on New Technologies, Mobility and Security (NTMS)*, 2026.
- [10] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions (SHAP)," *Conference on Neural Information Processing Systems*, 2025.
- [11] R. Jordaney, K. Sharad, S. K. Dash, Z. Wang, D. Papini, I. Nouredinov, and L. Cavallaro, "Transcend: Detecting Concept Drift in Malware Classification Models," in *Proc. 26th USENIX Security Symposium*, 2025.
- [12] A. P. Felt, E. Ha, S. Egelman, et al., "Android Permissions and Extension Architectures: User Attention and Behavior," in *Proc. 8th Symposium on Usable Privacy and Security*, 2023.
- [13] Google LLC, "Google Safe Browsing API v4 — Developer Guide," *Google Developers*, 2023. [Online]. Available: <https://developers.google.com/safe-browsing/v4>
- [14] V. Le Pochat, T. Van Goethem, S. Tajalizadehkhoob, M. Korczyński, and W. Joosen, "Tranco: A Research-Oriented Top Sites Ranking Hardened Against Manipulation," in *Proc. NDSS*, San Diego, CA, USA, 2019.
- [15] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proc. 22nd ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining*, pp. 785–794, 2016.
- [16] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image Quality Assessment: From Error Visibility to Structural Similarity," *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, Apr. 2004.
- [17] G. Wood, "Ethereum: A Secure Decentralised Generalised Transaction Ledger (Yellow Paper)," Ethereum Foundation, 2014. [Online]. Available: <https://ethereum.github.io/yellowpaper/paper.pdf>
- [18] S. Abdelnabi, K. Krombholz, and M. Fritz, "VisualPhishNet: Zero-Day Phishing Website Detection by Visual Similarity," in *Proc. ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pp. 1681–1698, 2020.
- [19] J. Chen, M. Duan, R. Zhang, L. Chen, B. Luo, and Y. Liu, "IoT Blockchain Consensus Strategy Based on IPFS and Practical Byzantine Fault Tolerance," in *Proc. 2018 IEEE Int. Conf. on Internet of Things (iThings)*, pp. 1040–1045, 2018.